



FIELDS

Insider Threat

Aaron Tronsgard, Kumar Sannidhya Shukla, Brinda Venkataramani

June 15, 2026

- 1 Introduction
- 2 Methodology (Part 1)
- 3 Methodology (Part 2) and Conclusion

Introduction

- We were approached by a large tech company for help with their threat alert system.
 - They have a mature insider threat stack.
 - Includes multiple models, IDS, and threat hunting methods.

Introduction

- We were approached by a large tech company for help with their threat alert system.
 - They have a mature insider threat stack.
 - Includes multiple models, IDS, and threat hunting methods.
- They receive too many alerts for their security analysts to go through manually, resulting in alert fatigue.



The data

- What does the data look like?

The data

- What does the data look like?
- The company's system flags suspicious events and records data about the event and the entities involved.

The data

- What does the data look like?
- The company's system flags suspicious events and records data about the event and the entities involved.

Sample Event:

```
"id": "224a7b52-a126-44bc-90d4-301ab4de4664",  
"type": "Authentication",  
"timestamp": "2025-05-30T21:55:58.061710",  
"severity": 6,  
"description": "Service account used interactively",  
"alert_style": "raw_telemetry",  
"alert_message": "Windows Event ID 4624: Service account used interactively | User: jsmith | Workstation: DC-252 | Source IP: 192.168.37.6"
```

The data

- What does the data look like?
- The company's system flags suspicious events and records data about the event and the entities involved.

Sample Event:

```
"id": "224a7b52-a126-44bc-90d4-301ab4de4664",  
"type": "Authentication",  
"timestamp": "2025-05-30T21:55:58.061710",  
"severity": 6,  
"description": "Service account used interactively",  
"alert_style": "raw_telemetry",  
"alert_message": "Windows Event ID 4624: Service account used interactively | User: jsmith | Workstation: DC-252 | Source IP: 192.168.37.6"
```

Sample Entity:

```
"id": "41d657b6-c392-4543-9568-dcd5b0666f8a",  
"type": "Database",  
"name": "products_db",  
"properties": {  
  "name": "products_db",  
  "type": "SQL Server",  
  "size": "474 GB",  
  "sensitivity": "Medium"
```


The data

- What does the data look like?
- The company's system flags suspicious events and records data about the event and the entities involved.

Sample Event:

```
"id": "224a7b52-a126-44bc-90d4-301ab4de4664",  
"type": "Authentication",  
"timestamp": "2025-05-30T21:55:58.061710",  
"severity": 6,  
"description": "Service account used interactively",  
"alert_style": "raw_telemetry",  
"alert_message": "Windows Event ID 4624: Service account used interactively | User: jsmith | Workstation: DC-252 | Source IP: 192.168.37.6"
```

Sample Entity:

```
"id": "41d657b6-c392-4543-9568-dcd5b0666f8a",  
"type": "Database",  
"name": "products_db",  
"properties": {  
  "name": "products_db",  
  "type": "SQL Server",  
  "size": "474 GB",  
  "sensitivity": "Medium"
```

Sample Relationship:

```
"source": "10b9d856-4586-4484-bf79-83878d28f8d7",  
"target": "d6e99e0e-7ca1-43c6-a3fe-f5a3f0e73520",  
"weight": 1.0,  
"type": "related_to"
```

Business Needs

- The data consists of hundreds of events, entities, and relationships between them.
- This is too much for the analyst to go through every week.

Business Needs

- The data consists of hundreds of events, entities, and relationships between them.
- This is too much for the analyst to go through every week.
- Our goal is to clump events together into potential attack scenarios and provide the analyst with a (much smaller) list of events to look into manually.

Business Needs

- The data consists of hundreds of events, entities, and relationships between them.
- This is too much for the analyst to go through every week.
- Our goal is to clump events together into potential attack scenarios and provide the analyst with a (much smaller) list of events to look into manually.



Outcomes

- Success will be measured by our ability to make the analyst's job easier.

Outcomes

- Success will be measured by our ability to make the analyst's job easier.
- Saving the company money and resources, mostly in human-hours, and allowing the analyst to spend more time on other projects.

Outcomes

- Success will be measured by our ability to make the analyst's job easier.
- Saving the company money and resources, mostly in human-hours, and allowing the analyst to spend more time on other projects.
- While also hopefully being able to more reliably alert the company to malicious attack scenarios.

Anomaly detection

- **Anomaly:** Anomalies are data patterns that have vastly different characteristics from *normal* data points.
- Anomaly detection provides critical and actionable information about the data in various domains:
 - Anomaly in credit card transactions → Fraudulent use
 - Anomaly in astronomical images → New celestial body
 - Anomaly in network traffic → Potential attack
- A naive solution: treat this as a (supervised) classification problem
- This creates a profile for all ‘*normal*’ data points and anomalies are those that do not conform to this profile.
 - Slower
 - Lots of false positive
 - Susceptible to swamping effect
 - Feasible only in low dimensions
- We will **Isolation Forests** for anomaly detection.

Isolation Forest

- Goal: detect anomalies in high-dimensional data
- Key idea: anomalies are “few and different”, whereas, *normal* data points are “many and similar”.
- Instead of modeling normal data, we isolate points
- Anomalies are those that isolate *easily*.

Core Idea

- Build random binary trees called **isolation trees**
- At each node:
 - randomly choose a feature
 - randomly choose a split value
- Anomalies tend to be isolated in fewer splits
- Normal points require deeper paths to isolate

Scoring Anomalies

- For each point x , compute average path length $h(x)$ over trees
- Define anomaly score:

$$s(x, n) = 2^{-\frac{h(x)}{c(n)}}$$

where $c(n)$ is average path length in a binary search tree

- High score $\Rightarrow$ more anomalous

Algorithm Summary

- Sample subset of data
- Build many isolation trees
- Compute path lengths for each point
- Aggregate to obtain anomaly score
- Flag points with high scores as anomalies

Key advantage: No distance or density estimation required

The Isolation Forest Algorithm

Algorithm 1 $\text{iTree}(X')$

Input: X' , Input data

Output: A tree

- 1: **if** X' cannot be divided **then**
 - 2: **return** $\text{exNode}(X')$
 - 3: **end if**
 - 4: $Q \leftarrow$ list of attributes in X'
 - 5: randomly select an attribute $q \in Q$
 - 6: randomly select a split point p between the **max** and **min** values of the attribute q in X'
 - 7: $X_l \leftarrow \text{filter}(X, q \leq p)$
 - 8: $X_r \leftarrow \text{filter}(X, q > p)$
 - 9: **return** $\text{inNode}(\text{iTree}(X_l), \text{iTree}(X_r))$
-

The Isolation Forest Algorithm

Algorithm 2 iForest(X, t, ψ)

Input: X data, t number of trees, ψ subsampling size

Output: A set of iTree's

- 1: Initialize Forest
 - 2: **for** $i = 1$ to t **do**
 - 3: $X' \leftarrow \text{sample}(X, \psi)$
 - 4: Forest $\leftarrow$ Forest $\cup$ iTree(X')
 - 5: **end for**
 - 6: **return** Forest
-

The Isolation Forest Algorithm

Algorithm 3 iForest(X, t, ψ)

Input: X data, t number of trees, ψ subsampling size

Output: A set of iTree's

```
1: Initialize Forest
2: for  $i = 1$  to  $t$  do
3:    $X' \leftarrow \text{sample}(X, \psi)$ 
4:    $\text{Forest} \leftarrow \text{Forest} \cup \text{iTree}(X')$ 
5: end for
6: return Forest
```

- Time complexity: $\mathcal{O}(t\psi^2)$
- Space complexity: $\mathcal{O}(t\psi)$

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - **events**

¹And perhaps some metadata

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - events
 - entities

¹And perhaps some metadata

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - events
 - entities
 - relationships

¹And perhaps some metadata

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - `events`
 - `entities`
 - `relationships`
- `events` is an array consisting of event objects which provide the details about the security incidents like timestamp, severity score and description.

¹And perhaps some metadata

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - **events**
 - **entities**
 - **relationships**
- **events** is an array consisting of event objects which provide the details about the security incidents like timestamp, severity score and description.
- **entities** is an array consisting of entity objects which provide the details of the affected entity (which could be a user, a host, a file, a process *etc.*) like the type of entity and some metadata about them (like name, IP address, department *etc.*).

¹And perhaps some metadata

The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - **events**
 - **entities**
 - **relationships**
- **events** is an array consisting of event objects which provide the details about the security incidents like timestamp, severity score and description.
- **entities** is an array consisting of entity objects which provide the details of the affected entity (which could be a user, a host, a file, a process *etc..*) like the type of entity and some metadata about them (like name, IP address, department *etc.*).
- **relationships** array gives us the links between events and entities and the '*weight*' of the link.

¹And perhaps some metadata

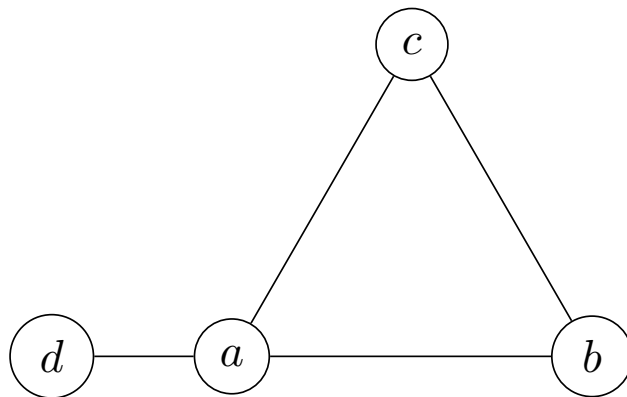
The Data

- The data consist of a single JSON file, consisting of 3 arrays ¹:
 - **events**
 - **entities**
 - **relationships**
- **events** is an array consisting of event objects which provide the details about the security incidents like timestamp, severity score and description.
- **entities** is an array consisting of entity objects which provide the details of the affected entity (which could be a user, a host, a file, a process *etc..*) like the type of entity and some metadata about them (like name, IP address, department *etc.*).
- **relationships** array gives us the links between events and entities and the '*weight*' of the link.
- How do we handle this data?

¹And perhaps some metadata

Graphs

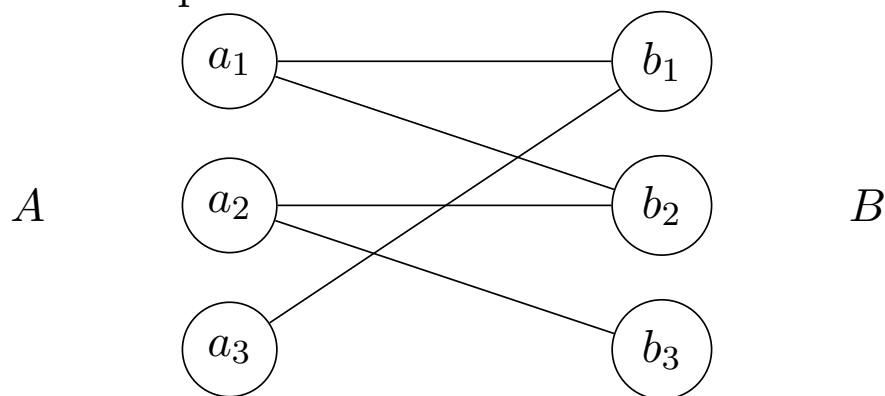
- A graph G consist of a pair (V, E) of vertices V and edges $E \subseteq \binom{V}{2}$.
- For example, the following graph



consists of the vertex set $V = \{a, b, c, d\}$ and the edge set $E = \{\{a, b\}, \{b, c\}, \{c, a\}, \{d, a\}\}$

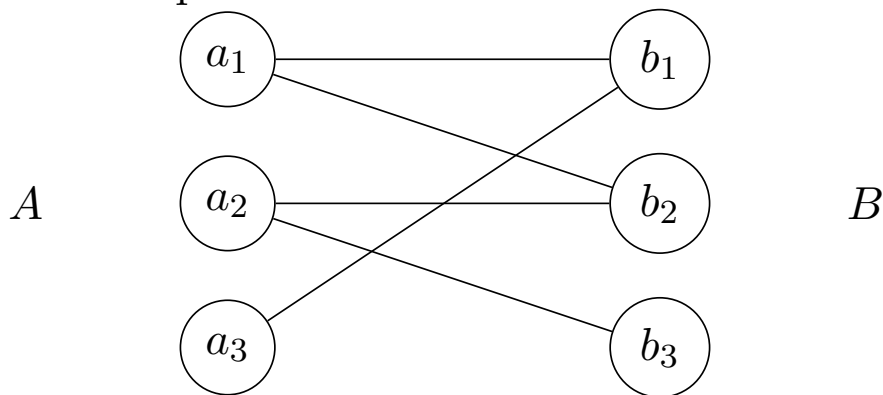
Bipartite Graphs

- A bipartite graph is a graph whose vertex set V can be partitioned $G = A \sqcup B$ such that no two vertices in A share an edge, and no two vertices in B share an edge.
- An example:



Bipartite Graphs

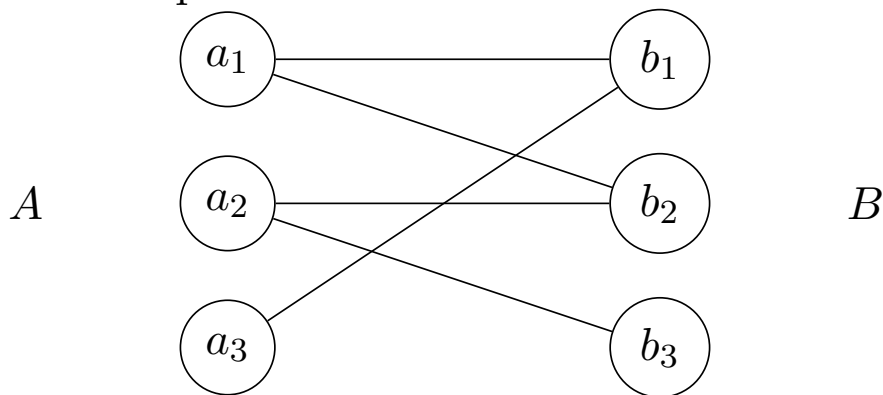
- A bipartite graph is a graph whose vertex set V can be partitioned $G = A \sqcup B$ such that no two vertices in A share an edge, and no two vertices in B share an edge.
- An example:



- What is a criterion to determine if a graph is bipartite?

Bipartite Graphs

- A bipartite graph is a graph whose vertex set V can be partitioned $G = A \sqcup B$ such that no two vertices in A share an edge, and no two vertices in B share an edge.
- An example:



- What is a criterion to determine if a graph is bipartite?
- No odd cycle $\iff$ Bipartite

How to handle the data?

- We build a bipartite graph out of the provided data as follows:

How to handle the data?

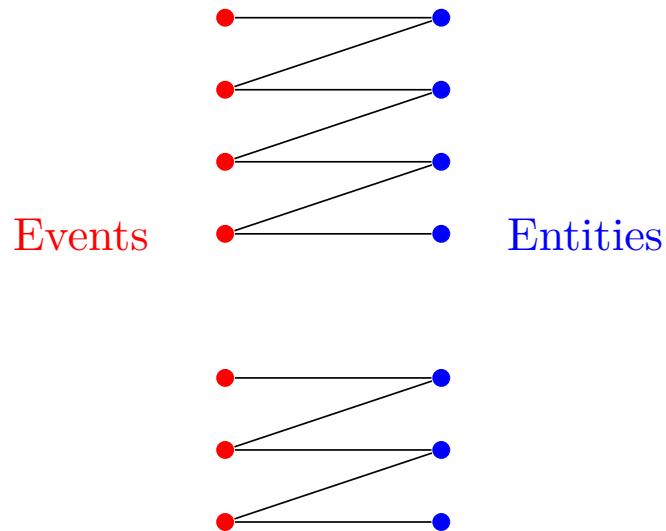
- We build a bipartite graph out of the provided data as follows:
- We let the vertex set be **events** $\sqcup$ **entities**.

How to handle the data?

- We build a bipartite graph out of the provided data as follows:
- We let the vertex set be `events` $\sqcup$ `entities`.
- We let the edge set be the `relationships`.

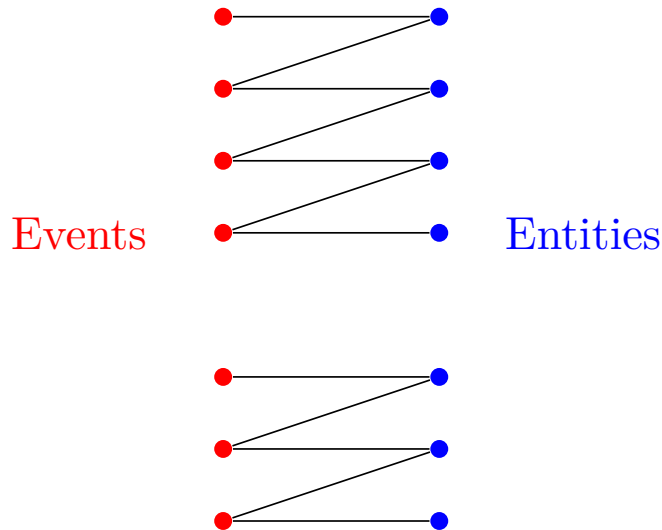
How to handle the data?

- We build a bipartite graph out of the provided data as follows:
- We let the vertex set be **events** $\sqcup$ **entities**.
- We let the edge set be the **relationships**.
- An example:



How to handle the data?

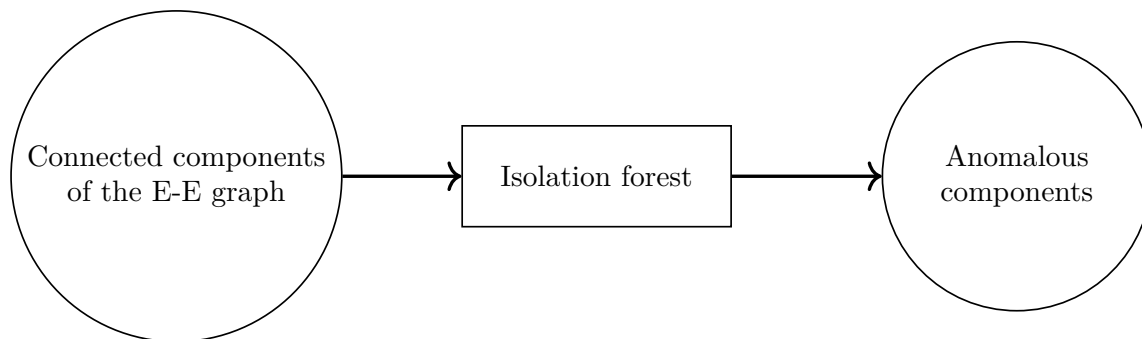
- We build a bipartite graph out of the provided data as follows:
- We let the vertex set be **events** $\sqcup$ **entities**.
- We let the edge set be the **relationships**.
- An example:



- Observe that this example has 2 connected components.

Isolating the anomalous components

- We feed the connected components of this graph to the isolation forest algorithm.
- The pipeline:



- The components with high anomaly score are likely to be attack scenarios.

Scoring and triage tiers

- Each subgraph is reduced to a fixed feature vector: mean edge weight, mean and maximum severity, kill-chain stage coverage, a *temporal-order* score, span, and size.
- The temporal-order score is the fraction of consecutive (time-sorted) events whose MITRE ATT&CK stage is non-decreasing. A genuine attack progresses in order — Initial Access → Privilege Escalation → Defense Evasion → Collection → Exfiltration — so it scores 1.0, whereas uncorrelated noise does not.
- An **Isolation Forest** scores every subgraph without using any labels. Its contamination parameter is set from the dataset's FP:TP ratio metadata, so the method ports directly to new input files.
- Subgraphs are then split into three tiers:
 - **Critical** — flagged by the model; the analyst must review.
 - **Elevated** — the next 5% by score; review as capacity allows.
 - **Low** — logged, surfaced only on demand.

Key performance indicators

- On the provided dataset, the pipeline ingests **1,012** raw alerts and decomposes them into **185** candidate subgraphs.
- Only **8** subgraphs (**43** events) reach the Critical tier — a **95.8%** reduction in the events an analyst must triage.
- All **6** ground-truth attack scenarios land in the Critical tier: **100% recall** at **75% precision**. The two false positives that accompany them are cheap — a few minutes of review each — whereas a missed attack is an incident.
- Widening to Critical + Elevated still cuts the workload by **90.8%** while providing a second, lower-priority queue.

Performance across datasets

- We ran the identical pipeline on the provided dataset and on two independently generated synthetic datasets (same schema, harder separation) that carry ground-truth labels.

Dataset	Events	Subgraphs	Recall	Precision
Provided	1,012	185	100% (6/6)	75%
Synthetic 1	998	179	83% (5/6)	62%
Synthetic 2	997	179	100% (6/6)	75%

- Recall is measured at the Critical tier. In **every** dataset, **all 6** attacks fall within Critical + Elevated — so no attack is ever pushed off the analyst's queue.
- The single Critical-tier miss (Synthetic 1) is a hard false positive that mimics a full kill chain; it still surfaces one tier down.

Validating without labels

- Most datasets ship **no ground-truth labels** — as in a real deployment, where the analyst is the one assigning them. We can still sanity-check the output two ways.
- **Against metadata.** The provided unlabeled set declares **3** expected attack scenarios in its metadata. With no access to labels, the pipeline flags **exactly 3** Critical subgraphs — the correct count falls straight out of the FP:TP-driven threshold.
- **Against structure.** The flagged subgraphs share an unmistakable profile that noise does not:

	Critical	Low
Avg. MITRE stages covered	5.0 / 5	2.1 / 5
Avg. edge weight	1.00	0.59
Avg. severity	7.6	2.5

- Every flagged subgraph spans the **full kill chain** with **maximal-**

Making it legible: subgraph visualization

- `visualize_graphs.py` renders each subgraph as a **bipartite graph**: event nodes (coloured by MITRE stage) on one side, the users, hosts, files, and domains they touch on the other.
- Edge thickness encodes the relationship **weight**, so a high-confidence attack subgraph is visibly denser and more strongly connected than a noise cluster — the analyst sees *why* it scored highly at a glance, with no need to read the raw JSON.
- The same tool plots the kill-chain timeline and the feature distributions, giving a fast, visual second opinion alongside the numeric score for any subgraph the analyst wants to inspect.

Ranking, not just flagging

- The tier threshold is a convenience, not a gate. Every subgraph receives a continuous anomaly score, and the system prints the **top-10 most anomalous** subgraphs regardless of tier.
- This matters because the threshold is deliberately conservative. An analyst scanning the ranked list can always look one or two places *past* the cut-off and investigate anything that stands out to them — a subgraph does not have to be formally flagged to be reviewed.
- Nothing is ever hidden: “Low” simply means low-priority, not discarded. The full ranked list, with the score explanation for each subgraph, remains available.
- In practice the genuine attacks occupy the very top of the ranking (ranks 1–6 on the provided data), so a human reading top-down reaches every real incident before any noise.

Manpower and cost saved (per batch)

- Industry estimates put average alert triage at roughly **7 minutes** per alert and a fully-loaded SOC analyst at about **\$67/hour** (~\$48/hr U.S. base $\times$ 1.4 for benefits and overhead).²
- Manually triaging all **1,012** alerts $\approx$ **118 analyst-hours** $\approx$ **\$7,900** per weekly batch.
- Reviewing only the **43** Critical-tier events takes **~5 hours** (~\$340) — a saving of **~113 hours** and **~\$7,600** per week.

Approach	Alerts	Hours	Loaded cost
Manual (everything)	1,012	118	\$7,900
Critical only	43	5	\$340
Critical + Elevated	93	11	\$730

²Triage time: Prophet Security, *SOC Capacity Modeling* (2025). Salary: Glassdoor / Salary.com (2026).

Manpower and cost saved (annualized)

- The provided dataset covers roughly **one week** of alerts. Extrapolating the same volume across a year ($\times 52$):

Approach	Analyst-hours/yr	Loaded cost/yr
Manual (everything)	$\sim 6,100$	$\sim \$413,000$
Critical only	~ 260	$\sim \$18,000$
Saved	$\sim 5,900$	$\sim \$395,000$

- That is roughly **3 full-time analysts** freed from manual triage — redeployable to deeper investigation and threat hunting — while **every known attack is still caught**.
- Figures are a first-order extrapolation from one week and assume a constant alert rate; they are intended to convey scale, not a budgeted forecast.

Future work: analyst feedback and retraining

- The current model is fully **unsupervised** — it needs no labels, which is what lets it run on any new input file on day one.
- A natural extension is a **feedback loop**. Each subgraph in the ranked output could expose a one-click tag — `TRUE_POSITIVE`, `FALSE_POSITIVE` or `NEEDS_INVESTIGATION` — written to a standardized log alongside the subgraph's feature vector.
- Because we already compute a feature vector per subgraph, every tag becomes a labelled training example essentially for free. Once enough accumulate, they support a **supervised** layer (e.g. logistic regression or a random forest) that complements the anomaly score and learns this organisation's specific notion of a true threat.
- Tags would also let the threshold **self-calibrate**: repeated confirmations just below the cut-off can nudge the contamination estimate automatically. This was scoped but left as future work.

Conclusion

- We turn a flat stream of **1,012** alerts into a ranked, tiered queue, cutting the analyst's triage load by roughly **96%** — on the order of **\$0.4M** and **3 analyst-FTEs** per year — while catching **every** known attack on the provided data.
- MITRE ATT&CK kill-chain **temporal logic** is the key signal: it distinguishes coordinated attacks from coincidental clusters of high-severity noise, and it is what carries performance over to harder, unseen data.
- The system is unsupervised at its core and ranking-based at the surface: nothing is hidden, the full ordered list is always available, and analyst feedback offers a clear path to a self-improving model.